

Vanliga fel

Det finns ett antal vanliga fel som nybörjare (och experter med för den delen!) inom C++ ofta gör. Några av dessa redogörs för här, för att göra det lättare att veta var felet skall sökas när operativsystemet ger felmeddelandet `segmentation fault` eller `General protection fault`. Problemet med fel inom dynamisk minneshantering är att de kan vara ganska svåra att hitta, eftersom de inte alltid direkt åstadkommer ett fel, utan ibland kan programmet fungera långa tider trots att ett fel redan uppstått. Sedan kommer en krasch på ett totalt oväntat ställe, kanske på ett ställe som inte ens hanterar dynamiskt minne överhuvudtaget.

Onitialiserat minne

Pekare som skall användas för att peka på allokerat minne kan inte avrefereras före de satts att peka på giltigt allokerat minne. Detta är ett ganska vanligt fel. Följande exempel leder troligtvis till problem:

```
float * Koefficient;  
  
// fel!  
*Koefficient = 1.03;
```

Vi har inte allokerat minne för `Koefficient`, inte har vi heller satt den att peka på något annat minnesområde eller någon annan variabel. Kom ihåg att allokera minne före du ämnar använda det!

Frigjort minne

Det är ett allvarligt fel att referera till minne som redan frigjorts. T.ex. följande exempel är felaktigt:

```
double * Buffer = new double [1024];  
...  
// frigör buffern  
delete [] Buffer;  
  
// fel!  
Buffer [10] = 34.09;
```

Här försöker vi referera till en vektor vi redan frigjort. I vissa fall kanske detta inte leder till ett omedelbart fel, utan felet kan uppstå senare, speciellt om det minnesblock som frigjordes har återallokerats på annat håll i programmet. I så fall uppstår *minneskorruption*, vilket kan vara *väldigt* svårt att hitta, eftersom felet visar sig på en helt annan plats än det uppstår.

Dubbel frigöring av minne

Allokerat minne får inte frigöras två gånger. Det är frågan om ett allvarligt fel. I ideala fall borde allokerat minne frigöras på endast ett ställe i programmet, men så är inte alltid fallet. Följande program är felaktigt:

```
double * Buffer = new double [1024];
```

```
...
// frigör buffern
delete [] Buffer;
```

```
....
// fel!
delete [] Buffer;
```

Vi frigör här `Buffer` två gånger. Man kan komma undan dessa problem genom att alltid nollställa pekare efter att minne frigjorts och även då de deklarerats. Följande program illustrerar metoden:

```
// initialisera pekare till 0
int * Block = 0;

// allokera minne
...

// kontrollera pekare före frigivning av minne
if ( Block != 0 ) {
    // vi har allokerat minne, frigör och nollställ pekare
    delete [] Block;
    Block = 0;
}
```

Ovanstående metod är mycket bra att använda om man frigör minne på olika ställen i ett program. Man kan då enkelt undvika att frigöra samma minnesblock två gånger.

Fel indexering av vektor

Detta är ett vanligt fel speciellt bland gamla Modula-2 och Pascal-programmerare. I dessa språk kan vektorers övre och undre index sättas separat, medan i C++ är en vektors undre index alltid 0. Man skall således indexera vektorer från 0 till *antalet element - 1*. Följande är en vanlig felsituation:

```
double * Buffer = new double [1024];

// iterera över alla element
for ( int Index = 0; Index <= 1024; Index++ ) {
    Buffer [Index] = ...;
}
```

Vi kommer här att indexera ända till element 1024, vilket är fel, 1023 är det sista elementet som tillhör vektorn. I dylika slingor bör man alltid fundera efter var man skall sluta iterera.

Minnesläckor

Minnesläckor är en annan typ av fel än de som visats ovan. De leder inte till en omedelbar terminering av programmet, utan de äter sakta upp resurser från operativsystemet, och leder förr eller senare till problem då minnet tar slut. I normala fall är små minnesläckor inget större problem, men om det är frågan om program som skall köras länge kan även små läckor leda till stora problem. Antag ett program som läcker minne 1kb (1024 bytes) per minut då det körs. Programmet läcker då 1440kb i dygnet (~1.4Mb). Om programmet skall snurra i dagar och veckor (vilket är normalt under Unix) läcker det 10080kb i veckan och ~43Mb i månaden. På några månader har programmet ätit upp minnet för de flesta maskiner. Antagandet här var att programmet läcker relativt lite minne, om det läckte 1kb i sekunden skulle det läcka ~90Mb i dygnet!

Minnesläckor uppstår då man har allokerat minne och sedan "tappat bort det" utan möjlighet att

kunna frigöra det. Följande exempel läcker minne:

```
int * Pekare = new int;  
Pekare = new int;
```

Vi har nu inget sätt att veta var vår första `int` finns, eftersom den enda pekare som fanns numera pekar på ett annat minnesområde. Följande är däremot korrekt:

```
int * Pekare1;  
int * Pekare2 = new int;  
Pekare1 = Pekare2;  
int * Pekare2 = new int;
```

Här har vi pekare som pekare på båda minnesområdena vi allokerat, och vi kan således frigöra båda minnesblocken om vi vill:

```
int * Pekare1;  
int * Pekare2 = new int;  
Pekare1 = Pekare2;  
int * Pekare2 = new int;
```

```
delete Pekare1;  
delete Pekare2;
```

[Förutgående](#)

Allokering och frigöring av
minne

[Hem](#)
[Upp](#)

[Nästa](#)
Minneshantering på C:s vis